

VRMoL システム設計書

1 システム概要

1.1 目的

VR空間内で記憶宮殿法を用いた記憶実験を行うためのシステムを構築する。

1.2 システム要件

1. 機能要件

- 10の異なる空間をVRで再現
- 空間オーディオと非空間オーディオの切り替え
- 単語カードの配置と管理
- 実験データの記録
- 進行状況の表示

2. 非機能要件

- パフォーマンス：72FPS以上を維持
- 遅延：音声遅延50ms以下
- 安定性：長時間の実験に耐えられる安定性
- 拡張性：新しいロケーションの追加が容易
- 保守性：コードの可読性と再利用性の確保

2 システム構成

2.1 ハードウェア要件

- **PC**：Windows 10/11、GPU RTX 3060 以上推奨
- **HMD**：Meta Quest 3（Air Link／Meta Link どちらでも可）
- **ヘッドホン**：AirPods Pro（Bluetooth・低遅延モード有効）

2.2 ソフトウェア要件

- **Unity**：2021.3 LTS
- **必須パッケージ**：
 - SteamVR Plugin 2.x
 - Steam Audio (v4.6)
 - Addressables

- Post-processing（URP 使用時は Volume）

3 詳細設計

3.1 ロケーション設計

3.1.1 基本仕様

- 各ロケーションのサイズ：5m x 5m x 3m（統一）
- ドアの位置：各ロケーションの端に配置
- 照明：各ロケーションに適した色温度と強度
- コライダー：壁、床、天井、家具に適切に設定
- モデル作成：Blenderで各部屋の基本モデルを作成
 - ポリゴン数：10,000以下
 - テクスチャサイズ：2048x2048以下
 - UVマッピング：適切に設定
 - エクスポート形式：FBX
 - スケール：1:1（Unity基準）

3.1.2 モデル最適化ガイドライン

- 静的バッチングのための適切なメッシュ分割
- LODの実装（3段階）
- コライダーの最適化
- テクスチャの圧縮設定
- マテリアルの最適化

3.1.2 ロケーション一覧

ID	テーマ	内装スタイル	特徴的な要素	環境音
01	和室	伝統的な和室	畳、障子、文机、掛け軸	カフェのざわめき
02	玄関	モダンな玄関	下駄箱、傘立て、鏡	森の小鳥のさえずり
03	廊下	シンプルな廊下	壁面装飾、照明	電車の走行音
04	リビング	コンテンポラリー	ソファ、テレビ、コーヒーテーブル	波の音
05	キッチン	モダンキッチン	システムキッチン、ダイニングテーブル	オフィスの環境音

ID	テーマ	内装スタイル	特徴的な要素	環境音
06	洗面所	モダンな洗面所	洗面台、鏡、収納	ジャズの演奏
07	ベランダ	開放的なベランダ	植物、洗濯物干し	地下鉄の駅の音
08	書斎	クラシックな書斎	本棚、デスク、椅子	カラオケの音
09	子供部屋	カラフルな子供部屋	ベッド、机、おもちゃ	オーケストラの演奏
10	自室	一般的な部屋	ベッド、机、クローゼット	市場の賑わい

3.2 音声システム設計

3.2.1 音声仕様

- フォーマット：WAV 48kHz/16bit
- チャンネル：モノラル
- 長さ：環境音は1分ループ
- 音量：-28 LUFSに統一

3.2.2 空間オーディオ設定

```
[System.Serializable]
public class AudioSettings {
    public bool isSpatial = true;
    public float spatialBlend = 1.0f;
    public float minDistance = 1.0f;
    public float maxDistance = 10.0f;
    public AnimationCurve falloffCurve;
}
```

3.3 インタラクション設計

3.3.1 単語カード

```
public class CardController : MonoBehaviour {
    [SerializeField] private float grabDistance = 0.5f;
    [SerializeField] private float throwForce = 5.0f;

    private bool isGrabbed = false;
    private Transform originalParent;
```

```

    public void OnGrab() {
        // グリップボタンで掴む
    }

    public void OnRelease() {
        // グリップボタンで離す
    }
}

```

3.3.2 ドア制御

```

public class DoorController : MonoBehaviour {
    [SerializeField] private float openAngle = 90f;
    [SerializeField] private float openSpeed = 2f;

    private bool isOpen = false;

    public void ToggleDoor() {
        // ドアの開閉
    }
}

```

3.4 データ管理設計

3.4.1 実験データ構造

```

[System.Serializable]
public class ExperimentData {
    public string participantId;
    public int locationId;
    public float timestamp;
    public Vector3 position;
    public string action;
    public string cardId;
    public bool isSpatialAudio;
}

```

3.4.2 データ保存

```

public class DataLogger : MonoBehaviour {
    private string savePath;
    private List<ExperimentData> dataBuffer;
}

```

```

    public void LogData(ExperimentData data) {
        // データの記録
    }

    private void SaveToCSV() {
        // CSVファイルへの保存
    }
}

```

3.5 UI設計

3.5.1 進行状況表示

```

public class ProgressUIDisplay : MonoBehaviour {
    [SerializeField] private TextMeshProUGUI locationText;
    [SerializeField] private TextMeshProUGUI progressText;

    public void UpdateProgress(int current, int total) {
        // 進行状況の更新
    }
}

```

4 実装ガイドライン

4.1 命名規則

- クラス名：PascalCase
- 変数名：camelCase
- 定数：UPPER_CASE
- メソッド名：PascalCase

4.2 コーディング規約

- アクセス修飾子を明示的に指定
- 適切なコメントの記述
- エラー処理の実装
- パフォーマンス最適化の考慮

4.3 アセット管理

- Addressablesを使用した動的ロード

- アセットバンドルの最適化
- メモリ管理の徹底
- Blenderモデルの管理
 - バージョン管理
 - テクスチャの最適化
 - マテリアルの一元管理
 - メッシュの最適化

5 テスト計画

5.1 単体テスト

- 各コンポーネントの機能テスト
- エラー処理のテスト
- パフォーマンステスト

5.2 統合テスト

- ロケーション間の遷移テスト
- 音声システムの統合テスト
- データ記録の整合性テスト

5.3 ユーザビリティテスト

- VR操作の直感性テスト
- 音声の定位テスト
- 全体的な体験の評価

6 品質管理

6.1 パフォーマンス指標

- FPS：72以上
- CPU/GPU Timings：13.8ms以下
- メモリ使用量：2GB以下

6.2 品質チェックリスト

- Quest 3の6DoF動作確認
- AirPods ProのAV同期遅延確認
- データログの完全性確認

- 各ロケーション間の音声漏れ防止確認

7 更新履歴

- 2024/06/03: 初期設計書の作成
- 2024/06/03: ロケーション構成の更新
- 2024/06/03: システム要件定義の追加
- 2024/06/03: Blenderモデル作成方針の追加